

SVG (8)
Testing (34)
The Ajax Experience (62)
TIBCO (24)
Tip (60)
Toolkit (165)
Tutorial (17)
UI (100)
Unobtrusive JS (20)
Usability (71)
Utility (148)
W3C (3)
Web20 (24)
Widgets (8)
Workshop (7)
XmlHttpRequest (39)
Yahoo! (98)

ARCHIVE
May 2008 (62)
April 2008 (110)
March 2008 (108)
February 2008 (113)
January 2008 (110)
December 2007 (92)
November 2007 (106)
October 2007 (123)
September 2007 (99)
August 2007 (109)
July 2007 (89)
June 2007 (86)
May 2007 (89)
April 2007 (87)
March 2007 (106)
February 2007 (87)
January 2007 (105)
December 2006 (92)
November 2006 (120)
October 2006 (120)
September 2006 (92)
August 2006 (106)
July 2006 (100)
June 2006 (83)
May 2006 (99)
April 2006 (78)
March 2006 (111)
February 2006 (117)
January 2006 (118)
December 2005 (106)
November 2005 (111)
October 2005 (67)
September 2005 (87)
August 2005 (85)
July 2005 (51)
June 2005 (70)
May 2005 (63)
April 2005 (11)
March 2005 (23)
0 (1)

One place that you could use this API would be on the server-side. For example, using [Ajotana Jaxer](#). Although, you could also interface directly to Java, or just use the Mozilla utilities directly.

Posted by *Dion Almaer* at 10:51 am

6 Comments

3 rating from 22 votes

Thursday, May 1st, 2008

Ajaxian Roundup for April, 2008: CSS goodness, Ext licenses, and the Cloud

Category: **Roundup** 

March has flown by for me, and we had some great announcements, and some busy threads of discussion to show for it. The Webkit folk have had the great insight to realize that although SVG and canvas are still thought of as more advanced technology and are not mainstream in anyway, the problems that they can solve are very useful. In fact, you can take those tools and give specific solutions to use cases. For example, round some corners! The [CSS animations](#) and [CSS masks](#) work are killer good and exciting.

March was a coming out party for the Cloud, with the technical preview of [Google App Engine](#) and the news of the upcoming [Ajotana Cloud](#). I have a feeling that 2008 will be the "when we get to hit the DEPLOY button" year for developers, and I am very excited about it!

Finally, the ugly parts. [Ext JS 2.1 was released](#), and with it came [a license change](#). This brought up the undercurrent of some in the community that thought that the old license wasn't valid, and with the GPL change we saw [OpenEXT, the fork](#).

The Ext JS team is [responding with open source exceptions, and is asking for community input](#).

Here is the full roundup:

JavaScript

- [Google offers Search, Feed, and Translation APIs to Non Ajax Usage](#)
- [Talking about JavaScript 1.7, 1.8 and 1.9 before we get to 2.0](#)
- [ECMAScript 4 Update: New Grammar, and No Tail Calls](#)
- [JavaScript, C#, and ExtSharp](#)
- [ASP.Net Ajax Site and Silverlight](#)
- [JavaScript SAX Based Parser](#)
- [window.crypto: want crypto primitives in the browser? You may already have it](#)
- [JavaScript has staying power: Used in Stargate](#)
- [JS Time Machine](#)
- [Pi.Debugger: Cross browser debugging](#)
- [A wishlist for Ajax APIs](#)
- [Java Plugin: The Kernel is back](#)
- [Dromaeo: JavaScript Engine Testing](#)
- [using.js: manage JavaScript dependencies](#)
- [Server-side jQuery, E4X, and more with Jaxer](#)
- [Joose expands with new ORM](#)
- [Event Delegation for blur and focus](#)
- [Three Helpful JavaScript Libraries](#)
- [Conform your JSON to ECMAScript 4 with JCON](#)
- [JavaScript: The Good Parts](#)
- [DOH, let me test my code!](#)
- [Coherent: Cocoa Databinding for Ajax](#)
- [Java in JavaScript](#)
- [100 Line Ajax Wrapper](#)
- [QraoWeb: Quicktime + JavaScript](#)
- [JavaScriptMVC Test Plugin](#)
- [Reminded of speaking your YAHOO! lang](#)
- [FancyUpload: Swift meets Ajax](#)
- [Making application modules communicate with each other using Decoupling](#)
- [Ajax Accessibility and ARIA](#)

jQuery

- [jQuery gets Classy](#)
- [jQuery: Java, GWT, and jQuery together](#)

Prototype

- [Timeframe: Prototype date range component](#)
- [gGallery: Prototype gallery application](#)
- [jQuery and Prototype Benchmarks](#)

Dojo

- [Usable Directory Listings with Dojo](#)
- [Dojo-Mini and the Feature Explorer](#)
- [Chandler Server Upgrades to Dojo 1.0.2](#)
- [Dojo 1.1 Nice Features](#)
- [Dojo XHR Plugins: How do you want your XHR today?](#)

Ext

- [Ext JS and the fun with Open Source licenses](#)
- [OpenEXT: The fork](#)
- [Ext JS responds with Open Source FLOSS Exceptions](#)
- [Ext JS 2.1 Released](#)
- [To GWT Ext or to Ext GWT?](#)

Moo

- [MooWheel: Unique Data Visualization](#)
- [NSFW Blocker Using MooTools](#)

Browsers / Standards

- [Gears and Web Standards](#)
- [What does the "Open Web" actually mean?](#)
- [The performance of your Acid 3](#)
- [IE 8 strict mode doesn't allow for CSS opacity?](#)
- [OpenAjax Call-to-Action for Browser Wishlist](#)
- [Browser Update: Firefox 3b5 and Opera Mini 4.1 beta](#)
- [IE 8 Security Updates](#)
- [Getting feedback to the IE 8 team](#)
- [Yahoo! BrowserPlus: The rumour is true](#)
- [Yahoo! BrowserPlus: Why wait for the news when you have strings?](#)
- [Last call for W3C XMLHttpRequest comments](#)
- [Embed your data- in HTML 5](#)
- [Taking Web Applications Offline, to the Desktop, and beyond](#)
- [Events Compatibility Tables: Powering the Dutch Royals](#)

to hear about any news that we could put up here.

We have individual blogs too:

- [Dion Almaer's Blog](#) (• [twitter](#))
- [Ben Galbraith's Blog](#)
- [Rob Sanheim's Blog](#)
- [Michael Mahemoff's Blog](#)
- [Chris Cornutt's Blog](#)
- [Dietrich Kappe's Blog](#)
- [Chris Heilmann's Blog](#)

FEEDS

Enter your Email

Powered by [FeedBlitz](#)




- [Are you sure your unload handler is firing in IE?](#)

CSS / UI

- [Fun with SVG and CSS Animations](#)
- [GChart 2.0: Give me some pie](#)
- [Canvas2Image: Save out your canvas data to images](#)
- [CSS Gradients in WebKit](#)
- [Using canvas to test your site with colorblind folk](#)
- [WebKit keeps going with CSS Masks](#)
- [CSS Variables are next?](#)

Mobile

- [iPhone WebKit Goodness: 3D CSS Transforms and ontouch events](#)
- [iPhone Remote Debugger](#)
- [Mozilla Fennec: The mobile browser wars](#)
- [Mobile Browser Concurrency Test: Get your mobile browsers ready](#)
- [Now your mobile phones get to take some Acid](#)

Performance and the Cloud

- [What does Google App Engine mean for Ajax developers?](#)
- [Aptana Cloud: Develop on your desktop, sync out to the cloud](#)
- [Appcelerator on App Engine](#)
- [Cuzillion: Performance best practices tool](#)
- [Fast Streaming Ajax Proxy](#)
- [What is the future of Ajax applications talking to the data tier?](#)

Showcases / Games

- [The Twubble with Rob Lee](#)
- [Super Mario: 14th of JavaScript](#)
- [Upside Down Text](#)
- [Mad Mimi: WYSIWYG Email Marketing](#)
- [Mosaic Image Builder with Ajax](#)
- [Immediate Transition and Mibbit](#)
- [Fingerprint: A prize for your typing](#)
- [THP: Your homepage can be a JavaScript command line](#)
- [JSONVid: Pure JavaScript Video Player](#)
- [ProtoRPG: Role Playing with Prototype](#)
- [Twistori: Telling a story with Tweets and Script.aculo.us](#)
- [Spoiler Blocker When JS Isn't Available](#)

Utilities / IDE

- [WaveMaker Visual Ajax Studio for Mac](#)
- [Spket IDE 1.6.11 Released](#)
- [NetBeans now includes JavaScript support](#)
- [DOMAssistant 2.7 is Out, Strong Unicode Support and Enhanced Performance](#)
- [JavaScript Type Inference in IDEs](#)

Misc / Humor

- [Google releases GWT for JavaScript 2](#)
- [Book review: Advanced Ajax by Lauriat \(Part 2 of 2\)](#)
- [But I'm not moving my mouse!](#)
- [Audible Ajax Episode 25: State of Ajax](#)
- [Popularity, History, and SCRIPT SHARED](#)
- [Flinching APIs in the Highlands](#)
- [Adobe Releases AIR for Linux, Joins LSF](#)
- [Web Archeology: Java Pluglet API](#)
- [SEO Rapper Friday](#)
- [Adobe AIR for JavaScript Developers Pocketguide](#)

If you have some news that you would like to contribute, [send us an email](#) or [tell us on Twitter](#).

Posted by [Dion Almaer](#) at 12:34 am

[Comment here](#)



Monday, April 28th, 2008

Aptana Cloud: Develop on your desktop, sync out to the cloud

Category: [JavaScript](#) , [Aptana](#) , [Cloud](#)



Aptana have [announced their cloud platform initiative](#). Aptana Cloud.

“Aptana Cloud plugs right into your IDE to provide instant deployment, smart synchronization, and seamless migration as you scale. Aptana Cloud is ideal for developers who use scripting languages to create Ajax, Facebook, mySpace and all other sorts of Web applications.

The key is that this isn't a infrastructure play, which they clearly point out:

“Aptana Cloud is architected to complement Cloud infrastructure providers like Amazon, Google, Joyent and others. To get started we've selected Joyent who serves up some of the largest of all Facebook apps.

This shows that their platform is designed to go meta, allowing you to deploy to various clouds in the future.

I think that the number one meme from Web 2.0 Expo last week was the "cloud", coming off of the excitement of Google App Engine. With Aptana Cloud we will see sophisticated tools to make us productive in the cloud. I am very excited to see that it won't be too long until developers will be able to build an application, hit DEPLOY, and be done. This is a huge win.

For developers:

- IDE plug-in integrates Cloud development, deployment and management life-cycles right into Aptana Studio in either its standalone or Eclipse based editions
- Instant deployment of projects to Cloud
- One click sync your project to the Cloud, or provide fine grained sync control too
- Integrated service management consoles
- Configure desired memory size and disk size
- Develop and instantly preview remote files right inside your Studio desktop environment
- Subversion source control.

As Ajax developers, the vision of **Jaxer** in the cloud is very interesting too. The entire application using JavaScript, and deployed up into the cloud, all through the nice IDE.

I was also pleased to read that not only Ruby on Rails, but Python is on the docket. After developing Django applications and playing with Google App Engine, I would love to be able to use Studio for Python code too. Not that [Emacs \(X or GNU\) isn't great. Steve!](#)

Darryl Taft wrote:

“Aptana adds extra value via IDE integration, deployment automation and active monitoring and notification services, Hakman said. “It’s like the ease and simplicity between iTunes on your desktop and its connectivity to services on the Web,” he said.

For developers, the IDE plug-in integrates cloud development, deployment and management lifecycles right into Aptana Studio in either its standalone or Eclipse-based editions, Hakman said. “The ability to deploy stuff to the cloud from Eclipse is part of this as well.”

Other developer features include instant deployment of projects to the cloud; one click can sync your project to the cloud or provide fine-grained sync control; the technology features integrated cloud services management; enables users to provision their cloud right from Aptana Studio, configure desired memory size and disk size, develop and instantly preview remote files right inside the Studio desktop environment, and includes Subversion Source Control.

Can't wait go get an invite. If you want one too, [request an account](#).

Also, Aptana Studio just passed 1,5 million downloads. Impressive.

Posted by *Dion Almaer* at 2:41 pm

[5 Comments](#)



Monday, April 14th, 2008

Server-side jQuery, E4X, and more with Jaxer

Category: [JavaScript](#) , [Server](#) , [Articles](#) , [Aptana](#)

Davey Waterson of the Aptana **Jaxer** team has posted an article on [using jQuery on the server-side with E4X and more](#) that shows an example of server-side Ajax with a popular framework.

The article describes a polling application that features:

- Using jQuery server-side to manipulate a DOM before it's sent to the client
- Doing some database / SQL interactions, server-side in javascript of course
- User sessions in javascript (**Jaxer**.session.set('status', status);)
- Using E4X on the server-side.

There are fun little features such as nuking portions of the page if the permissions call for it:

PLAIN TEXT

JAVASCRIPT:

```
1.
2. $( ( (status == 'voter') ? '.nonvoter' : '.voter' ).remove()
3.
```



Since the application delivers no JavaScript itself, this would all work even if the user has JavaScript turned off, on a simple mobile phone, etc.

Posted by *Dion Almaer* at 8:45 am

[4 Comments](#)



Tuesday, April 1st, 2008

Ajaxian Roundup for March, 2008: IE 8, Acid3, and Performance

Category: [Roundup](#)

As we sit through the fun and frolics of April fools day on the Internet, we can look back on a busy March in Ajax land. Often life is dominated by Ajax libraries, as they continue to make our lives easier, but this time it was about the browsers and the standards. With IE 8 we [finally saw a dusting off](#) of a browser that has been largely out of the game for many years. After poking around more with activities and Web slices, they actually seem interesting indeed, but we really care about the non-user facing stuff. We want IE 8 to catch up and give us our APIs. The first public beta is promising, and now we need to watch for the next. Safari 3.1 was [released](#), and Firefox 3 is [looking close](#), with the betas looking [top notch](#). I personally wouldn't mind teaching the browser a couple of [new tricks](#) too.

Standards were in the air starting with the [IE 8 standards mode switch](#) and then jumping to the [Acid 3 news](#), and even IE got [some Acid groove](#).

Finally, we seemed to have a ton of news revolving around performance. Some of this came from testing the new browsers and seeing huge performance improvements, but others were just new general best practices. Performance seems to be on the forefront of peoples minds right now, which is great to see.

Thanks for a great month. Please [contact us with Ajax news as you see it](#), and here is the roundup:

Browsers

- [Microsoft changes IE 8 defaults to be "standards mode"](#)
- [Silverlight Offline and AIR](#)
- [Acid 3 Ships: WebKit praised](#)
- [IE 8: Better Ajax, CSS, DOM, and new features](#)
- [WebKit, GTK, and Qt](#)
- [IE 8 on Acid](#)
- [Me dium shows off new IE 8 features](#)
- [Mozilla Prism update makes it easy to create wrappers](#)
- [Internal IE-HTML DOM still isn't XHTML compliant](#)
- [Firefox 3 beta 4: postMessage and perf](#)
- [Firefox 3 Memory Usage](#)
- [Safari 3.1 Released](#)
- [IE8 and Safari 3.1 compatibility updates](#)
- [Key events and Safari 3.1](#)
- [Acid 3: Opera Passed?](#)
- [WebKit Passes Acid3](#)
- [Will IE8 & HTML5 Make RSH Irrelevant?](#)
- [Where is Firefox on Acid 3? Here.](#)

- [Do you want your browser to Jabber away?](#)
- [Why getBoundingClientRect is important](#)
- [OpenID and OAuth in the browser?](#)

JavaScript

- [Generics in JavaScript 2](#)
- [Playing with language: Inlined fromMaybe](#)
- [Json.NET 2.0](#)
- [iWebMvc: DWR, Dojo, Spring and Hibernate/JPA](#)
- [JavaScript WebDAV Client](#)
- [Appcelerator: RIPE - SOA](#)
- [Controller: Event Delegation Library](#)
- [ClientSide Mootools Library Update](#)
- [Secrets of JavaScript Libraries](#)
- [ECMAScript 4 Progress Tracking](#)
- [Google Visualization joins the Ajax APIs](#)
- [Subclassing and the prototype chain](#)
- [JavaScript Metaclass Programming](#)
- [Echo 3 releases client side component model](#)
- [Google AJAX Traversal API](#)
- [Nitobi Survey Results on Ajax Development](#)
- [JavaScript Has Fashion and Luck to Thank?](#)
- [IE and WebKit Performance: Is WebKit the Ralph Nader of Browsers?](#)
- [Multiple File Uploads with Aptana Jaxer](#)
- [Helpful JavaScript Date Operations](#)
- [Using a hash property for security and caching](#)
- [Composing DSLs in JavaScript](#)

Dojo

- [Dojo 1.1 Released](#)
- [Dojo on AIR shows detail on AIR itself](#)
- [Dojo and Django templates on the server side with Jaxer](#)
- [Getting some \\$ with Dojo](#)
- [dojox.analytics for developers and more](#)
- [Dojo Storage updated for 1.0](#)

DWR

- [DWR 3.0 Features: Interview with Joe Walker](#)

Ext

- [Using the YouTube API via Ext](#)
- [To ExtPHP, or to PHP-Ext?](#)
- [Ext-based Jabber Client](#)

Prototype

- [ProtoSafe Eases Compatibility for Prototype](#)

jQuery

- [jQuery: extending jQuerys testrunner to all](#)
- [JavaScript and jQuery Talk](#)
- [Browser CSS float error detection with jQuery](#)

CSS / Design

- [CSSJanus: simple tool for RTL](#)
- [cvi_text_lib: cross API text stroking](#)
- [Progressive Enhancement with CSS support](#)

Performance

- [Backbase tests browser JavaScript and Render performance including IE 8](#)
- [IE 8 and Performance](#)
- [IE 8 Connection Parallelism Issues](#)
- [Lessons learned from improving Google Code web site performance](#)
- [Roundup on Parallel Connections](#)
- [The importance of bandwidth versus latency](#)
- [Yahoo! releases new performance best practices](#)

Tools / Utilities

- [Firecookie: Put your hand in the cookie jar with Firebug](#)
- [How green is your Web site?](#)
- [RadRails 1.0 Released](#)
- [Help Debug Firebug](#)
- [TraceTool: Now with open source JavaScript client](#)

iPhone / Mobile

- [iPhone SDK: Great if you like Cocoa, but what about us?](#)
- [iPhone SDK for Web Developers](#)
- [CUI1: CNET iPhone UI](#)
- [iPhone Optimization Script](#)
- [Google Gears for Mobile Released](#)

Databases

- [CouchDB: Using F4X to get the XML back](#)
- [Taffy DB Javascript Database](#)

Showcases

- [The FireEagle has landed - personal location information for your applications](#)
- [ProtoFlow: Coverflow for Prototype](#)
- [YTranscript: Using the brand new YouTube chromeless, scriptable player](#)
- [WiseMapping: More Ajax Mind Mapping](#)
- [AsciiFly: ASCII art library](#)
- [An Ajax Ascii Art Generator](#)
- [Snake](#)
- [Adobe Photoshop Express Launches](#)
- [markItUp! - Lightweight Text Editor](#)

Comet

- [pl.comet: simple comet library](#)
- [Comet Roundup: Gazing, Battles, and Protocols](#)

Posted by *Dion Almaer* at 9:00 am
[1 Comment](#)



Friday, March 28th, 2008

Dojo 1.1 Released

Category: [Dojo](#) , [Announcements](#)

The Dojo team has [released version 1.1](#) which includes from over [800 improvements](#):

- An easy to use and significantly improved [Dojo API Viewer](#) with some seriously great features, including the ability to easily find the original definition of a method that is "mixed-in"
- A growing collection of [demos, tutorials, and articles](#)
- A new BorderContainer Dijit, which is a much better way to handle layout-based widgets than SplitContainer and LayoutContainer
- Significant performance improvements to dojo.query and dojo.fx
- Support for [Adobe AIR](#) and [Jaxer](#), and updated [dojox.flash](#) and [dojox.offline](#) APIs
- Major improvements to Dijit infrastructure and widgets
- All around Dijit theme improvements including the CSS structure for themes, refinements to the Tundra theme, re-introduction of the Soria theme, and the newly added Nihilo theme
- DTL, the Django Template Language, is now available for use in widgets with dojox.dtl
- Vector graphics animations
- Additions to Dojo including an analytics package
- Improvements to Dojo Data and RPC, and [support for JSONPath](#)
- Many improvements to the build system including CSS optimization, multiple versions of the Dojo Toolkit co-existing in the same document, and other great tools for optimizing performance

What is next for Dojo 1.2?

“ On to 1.2, where the focus will be on continuing to refine Dijit, the Dojo Grid, Dojo Charting, a better approach to DojoX, and much much more.

Posted by [Dion Almaer](#) at 9:33 am
[13 Comments](#)

 4.4 rating from 70 votes

Thursday, March 20th, 2008

Multiple File Uploads with Aptana Jaxer

Category: [JavaScript](#) , [Examples](#) , [Aptana](#)

Dealing with file uploads can be a test of a Web framework. I personally long for the input type="file" to be improved with items such as multiple="true" for multiselection, as well as showing the status of the upload (20% complete).

The [Jaxer](#) folks have posted on [Easy File Uploading using Aptana Jaxer](#) which shows how you can tinker in JavaScript to get everything you need in a very simple way:

“ To receive the data from the form when submitted we put some [Jaxer](#) code into the page the form will be submitted to. The code below should be in script block with a runat = 'server' attribute, which makes the code run serverside and doesn't present it to the client so you don't expose any serverside filenames or folder structures.

```
PLAIN TEXT
HTML:
1.
2. <script type='text/javascript' runat='server'>
3. var message = "";
4.
5. for (fileCount=0; fileCount <Jaxer.request.files.length;
6.     var fileInfo = Jaxer.request.files[fileCount];
7.
8.     var destinationFilePath = Jaxer.Dir.resolvePath(file
9.     fileInfo.save(destinationFilePath);
10.
11.     message += "<br>" + [
12.         "Saved to : "           + destinationFilePat
13.         , "original filename : " +
14.         , "temp filename : "    + fileInfo.t
15.         , "contentType : "     + fileInfo.
16.         , "size : "             + fileInfo
17.     ].join("<br />");
18.
19. }
20. document.write(message);
21. </script>
22.
< _____ >
```

Posted by [Dion Almaer](#) at 5:52 am
[4 Comments](#)

 4.1 rating from 14 votes

Friday, March 7th, 2008

DWR 3.0 Features, Interview with Joe Walker

Category: [Ajax](#) , [DWR](#)

SitePen's [Dylan Schiemann](#) has [posted about](#) the recent [InfoQ interview of Joe Walker](#), and the upcoming release of DWR 3.0. The newest features for DWR include:

- Offline Support (Google Gears and/or Dojo Offline)
- TIBCO General Interface integration
- Aptana [Jaxer](#) integration
- OpenAjax Hub, PubSub, Bayeux, etc.

Joe gave a nice example of how the offline functionality could work:

“ For example, InfoQ uses DWR. If we get it right, it should be easy for you to make it so that if the network dies while a user is writing a comment, then the comment doesn't get lost. When the network is next up, and the user visits InfoQ, the comment will be resent then. It's a cool feature, and using DWR it should come almost for free.

DWR 3.0 is set to be released in June.

Posted by [Rey Bango](#) at 6:38 am
[Comment here](#)

 4.3 rating from 28 votes

Tuesday, March 4th, 2008

Dojo and Django templates on the server side with Jaxer

Category: [Dojo](#) , [Aptana](#)

Yesterday [we posted about Dojo and AIR](#) and how the framework could be well suited for certain desktop applications.

Today we have Krisztyan [talking about their Jaxer support](#) and how you can use the [Django template language](#), that Dojo recently added, to once again do its thing on the server side.

Kris tells us more:

```
PLAIN TEXT
HTML:
1.
2. <script runat="both">
3.   djConfig = {baseUrl:"/dojo/",usePlainJson: true, parseOn
4. </script>
5. <script runat="both" type="text/javascript" src="../../
6. <script runat="server">
7.   dojo.require("dojo.jaxer");
8.   ...
9. </script>
10.
```

“ Once this is done, Dojo should load in **Jaxer**, and you can utilize the library capabilities of the Dojo Toolkit on the server side. In particular, you can now use the DTL renderer as you would on the browser. The DTL renderer can take templates written using Django template language and render the templates based upon JSON data. If you are running **Jaxer**, you can view a demonstration of DTL rendering on the server by loading [/dojox/dtl/demos/demo_Templated_Jaxer.html](#) (make sure the Dojo Toolkit base URL is correct).

By using **Jaxer**'s capability to allow scripts to execute on both the client and server, we can utilize our Dojo Toolkit in both environments. On the server we can utilize many of the Dojo Toolkit's features such as functional language features, math libraries, cryptography, dates, encoding, and more to improve development speed and code quality. We can use the same powerful features and idioms in both the browser and the server. An example of a practical way to use the Dojo Toolkit in both environments is to utilize the DTL renderer as described above to generate HTML on the server, and then use the Dojo Toolkit's Ajax capabilities to dynamically update the page once it's on the browser. For example: we could use the DTL renderer to display the comments in a blog, dynamically add any new comment the user makes at the end of the list of comments, and then send the comment to the server using Ajax.

It is important to note that **Jaxer** is not capable of transferring the programmatically set event handlers for widgets—it can only send the static HTML to the browser. This means you can use DTL as a templating engine to create HTML on the server, but the Dojo Toolkit client side widgets are still necessary if you want to use interactive widgets on the browser.

Posted by [Dion Almaer](#) at 7:58 am
[Comment here](#)

★★★★☆
4 rating from 16 votes

Wednesday, February 27th, 2008

Kevin Hakman joins Aptana

Category: [Editorial](#) , [TIBCO](#) , [Aptana](#)

When we posted the last [podcast on Aptana Jaxer](#), someone commented on the fact that Kevin Hakman was there, and "Doesn't Kevin Hakman work for Tibco? What does he have to do with Aptana?".



Aptana has now come out with the news that [they have hired Kevin](#):

“ We are excited to announce that Kevin Hakman has joined Aptana to head up marketing and developer community programs. Kevin is a recognized leader in the Ajax community. Long before the term "Ajax" was coined, Kevin was pioneering single page web application concepts via the company he co-founded, General Interface Corp., one of the first enterprise Ajax libraries and visual development tools companies. General Interface Corp. went on to be acquired by TIBCO Software in 2004 as a complement the company's service-oriented architecture (SOA) products. Today TIBCO General Interface is used by many Fortune-scale companies for rapid Web application development and deployment atop their XML and SOAP data sources.

In addition to his historic role on the steering committee of the OpenAjax Alliance, Kevin currently chairs the organization's integrated development environment working group. The OpenAjax Alliance IDE Working Group which includes Adobe, Aptana, the Eclipse Foundation, IBM, Microsoft, Sun, TIBCO and others is now close to delivering a draft specification for a uniform way to describe Ajax libraries and controls and thus streamline the ability to use Ajax libraries within your development tools of choice. Kevin is a frequent speaker at Ajax industry events and is an author to many published articles on Ajax in the enterprise.

I asked Kevin a couple of questions about his move, comparisons between the companies, and the industry in general.

What similarities / differences do you see between pioneering heavy JavaScript on the client w/ GI and JavaScript on the server with Aptana

GI and Aptana's primary application models are different to serve different purposes similar to the way that Java has multiple presentation tier architectures for different kinds of apps: Swing, SWT, AWT for application GUIs and JSP, JSF for generating web pages. With GI, the concept in 2001 when we deployed some of the first "Ajax" apps pretty much evolved from a need to migrate software-style GUIs to the Web and a technical approach summarized as "we like Java Swing, but not the JRE dependency, so let's do something Swing-like in JavaScript with a JavaScript VM of sorts and get data as XML / SOAP via the XML HTTP Request Object -- an oh yeah, make it Visual Basic-like easy to visually develop too". That led to what I call a "Client-Services" architecture where you have a full fledged

JavaScript applications running in a web page talking to back-end data services plus the WYSIWYG rapid development tools for which GI has been recognized as best in class in [enterprise IT journals](#). With GI, you never really interact with the HTML page or its DOM. Instead there's a higher abstraction of application concepts and a model of the GUI, called "dual DOM" by some. The result is that the JavaScript applications are deployed into a webpage more like an applet, (except it's all in the same JavaScript memory space and there's no JRE dependency of course). This "Client-Services" architecture has been a great fit for many enterprises pursuing service-oriented architecture strategies.

Aptana Studio is clearly among the best-in class for the predominant model of Ajax development called "single DOM" where you work directly in the HTML DOM to describe elements within a page, then apply Ajax and CSS concepts to bring that page to life with interactive features. Aptana Studio meets the needs of Ajax developers working in this model extremely well. Whereas tools for web page development have historically focused on layout, image placement, styling, and JavaScript for roller coaster and simple navigational assists, Aptana saw that with Ajax and the popularity of all the single-DOM model Ajax libraries like dojo, Ext, prototype, MooTools and jQuery, the page was becoming increasingly programmatic in the client and needed tooling optimized for the programmatic page.

What excites you about Aptana

Aptana Jaxer is a very cool concept for all the JavaScript developers out there because they can now go boldly where no JavaScript developer has gone before-- to the server-side. Sure Netscape had LiveWire back in the day, but remember there was no real DOM, no real CSS and certainly no XMLHttpRequest at that time. Aptana **Jaxer's** genius is that it takes the same Mozilla engine we know and love in the browser, and puts it inside a server along with a bunch of handy **Jaxer** methods and properties to do server-side things like interact with databases, web services, session concepts, sockets and more. I personally love the ability to write a script that runs on the server, but call it from the client as if it were running on the client. In this case **Jaxer** handles all the sync or async communications for you transparently, and soon will provide end-to-end debug capabilities as well. We're also now working with Joe Walker of the DWR project to extend this kind of capability to remote Java objects as well through **Jaxer**. We're also investigating how to best interoperate with Microsoft's .NET platform.

To some, JavaScript is not a preferred language and that's where things like GWT and DWR come in real handy. At the same time there's millions of developers who like the ease and readability of JavaScript. Aptana Studio, with its JavaScript debug and type inference capabilities, and Aptana **Jaxer** with its server-side power enable all JavaScript developers to do much more, more quickly in the language we love to work with -- JavaScript! Thinking beyond the solid JavaScript programming tools in Aptana Studio today, things like visual WYSIWYG GUI composition and data-binding are clearly natural extensions. Not many people know that the Aptana team contributed to much of the Eclipse Monkey code which enables JavaScript to run within and communicate with Eclipse's APIs. This means that Aptana is extraordinarily well positioned to execute a first-class solution for WYSIWYG editing -- and further streamline the creation of Ajax web pages and full-featured Ajax applications.

Now you have been around JavaScript for awhile, what are you likes and dislikes, and do you have any thoughts on JavaScript 2?

Having been part of Ajax projects where we had single HTML pages running for 9 hours sessions with over 180 separate application modules you could load/unload during that time (and that was in 2001), the GI core team (Luke Birdeau, Michel Peachey, Jesse Costello Good, and myself) has had loads of experience in what it means to make highly scalable, full featured apps in JavaScript. Much of what we had to invent and implement in JS1.5 for the TIBCO GI Framework, things like object orientation for example and better memory management, is on the horizon for future releases of JavaScript. The primary pain of JS though, until recently, has been lack of great debugging, type inference, and code assist capabilities. What few people realize about Aptana Studio is that it not only code assists on the Ajax libraries you've loaded, it assist with the Ajax objects, classes, packages, functions and properties that you create too -- again a concept borrowed from the Java community, but with less programmatic overhead since its JavaScript.

Posted by *Dion Almaer* at 7:16 am

[8 Comments](#)

3.8 rating from 17 votes

Monday, February 25th, 2008

Adobe AIR v1.0 & Flex 3.0 Released; New Adobe Open Source Site Launched

Category: [Flash](#), [Ajaxian.com Announcements](#), [Adobe](#)

Continuing their march into the RIA space, Adobe announced today the official release of [AIR v1.0](#) and [Flex 3.0](#).

Adobe has taken the beta off of the wrapper as their have released both [AIR 1.0](#) and [Flex 3.0](#).

As Ajax developers, Adobe is trying hard to get us developing applications, not just Flash folks. They have a [place for us to start with AIR](#):

“The new Adobe AIR runtime enables Ajax developers to build rich Internet applications (RIAs) that deploy on the desktop. AIR applications run across operating systems on the WebKit HTML engine and are easily delivered using a single installer file. With Adobe AIR, Ajax developers can use their existing skills and code to build responsive, highly engaging applications that combine the power of local resources and data with the reach of the web.

Adobe AIR

The [AIR runtime and SDK](#) has gone through an especially long beta cycle (since June 2007) to ensure that both security and compatibility with existing frameworks was achieved. Some key new and/or updated features include:

- **Enhanced Desktop Functionality:** Drag and drop to the operating system, copy and paste between applications, launching of AIR applications from the desktop or the browser, and run in the background with notifications.
- **Data Access:** Adobe AIR now provides both synchronous and asynchronous access to the local file system, as well as structured data within a local database. This database is implemented using the open source SQLite database.
- **JS Library Support:** Most major Ajax frameworks can be used to build AIR applications. Supported frameworks include [jQuery](#), [Ext JS](#), [Dojo](#), and [Spry](#). Adobe AIR integrates JavaScript and ActionScript to allow cross-scripting



between the two languages, and integrated rendering of Flash and HTML content.

- **Security:** Applications built on Adobe AIR can only be installed through a trustworthy installation process that verifies that the application is signed via industry standard certificates, providing users with information about the source and capabilities of the application.

Flex 3.0

Adobe's Flash-based RIA development platform, Flex, continues to mature and has been picking up steam in both the corporate space as well as sites such as [blist](#) and [Scrapblog](#) who have embraced Flex whole-heartedly. Some of the new features in Flex 3.0 include:

- Intelligent coding, interactive step-through debugging, and visual design of user interface layout
- New capabilities for creating RIAs and building applications on Adobe AIR
- Integrates with Adobe Creative Suite® 3 making it easy for designers and developers to work together more efficiently.
- New testing tools including memory and performance profilers and integrated support for automated functional testing, speed up development and lead to higher performing RIAs.

One of the most compelling parts of the Flex announcement is the fact that Adobe has released the Flex SDK under the open source Mozilla Public License.

Adobe Open Source Site Launch

Finally, Adobe announced the launch of their new [Adobe Open Source](#) site which aims to "presents the definitive view into open source activities at Adobe, including details regarding projects that Adobe participates in and hosts."

“The new...website is designed to keep you up to date on Adobe open source activities, within Adobe as well as with the larger world. It will also be the point of entry to our source code contributions, including Flex, BlazeDS and others. We'll post news items, tell you where to see us, and keep you in touch with some of our favorite bloggers.

Currently, the site houses the [Flex SDK](#), [BlazeDS](#) and [Tamarin](#) projects, all of which have been open-sourced by Adobe.

Aptana has coordinated the release of their [AIR plugin](#) that includes support for **Jaxer** which allows you to write AIR apps that run on the desktop that include server-side code, written in JS, that can run on your backend server.

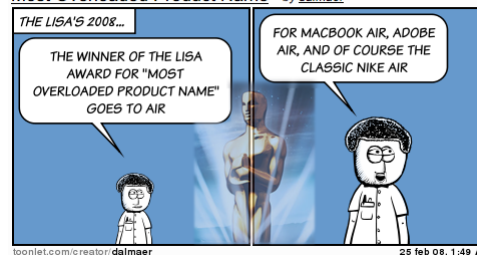
Adobe also put together a list of [featured applications](#) that you can check out.

Hitting a "1.0" release is a big deal (as is a 1.0.1 ;)), so congratulations to the entire Air team. Adobe is working hard to raise the bar in the RIA space by giving developers more tools with great functionality. 2008 is panning out to be an interesting year in web development.

Ben and I are at Adobe Engage today, and hope to find out more about Adobes plans in the coming year. We are [live twittering](#) using the [#engage hash tag](#).

To end with something a little fun, and since it was the Oscars tonight:

Most Overloaded Product Name by dalmaer



NOTE: Rey and I both wrote a post on this big release. This post is a conjoining of both posts into one

Posted by Dion Almaer at 2:02 am
[7 Comments](#)
4.5 rating from 29 votes

Monday, February 18th, 2008

Audible Ajax Episode 24: Aptana Jaxer Talk

Category: [JavaScript](#) , [Podcast](#) , [Framework](#) , [Aptana](#)

I had the opportunity to sit down with three fine gents from Aptana to discuss their recent launch of **Jaxer**, the "server side Ajax framework".

Paul Colton, Uri Sarid, and Kevin Hakman all sat with me to chat about things. I have already played with **Jaxer**, and created the Google Gears wrapper which can be used seamlessly for use cases such as "If the user doesn't have Gears installed, just do it on the server".

We discussed a lot in the twenty odd minutes including:

- Where the idea for **Jaxer** came from
- The difference between a server side JavaScript framework and **Jaxer** (since there are many of them!)
- How **Jaxer** works (think of a headless Mozilla browser)
- Side effects of going this direction
- How developers are using it
- How does your architecture change if you are using **Jaxer**?
- How can you talk to code in Java and other languages?
- How JavaScript 2 fits into the picture
- What about deployment?

A lot of good stuff. Thanks to the crew for taking the time to chat with me. What other questions do you have for them?

We have the audio [directly available](#), or you can [subscribe to the podcast](#). We also have the video in [high def here](#), or in normal def right below:

Posted by [Dion Almaer](#) at 9:37 am
7 Comments

★★★★☆
4.5 rating from 20 votes

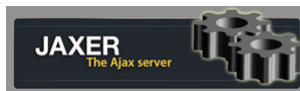
Wednesday, February 6th, 2008

Server Side JavaScript Database Access

Category: [JavaScript](#), [Gears](#), [Aptana](#)

Reposted from [my personal blog](#).

As soon as I started to play with [Aptana Jaxer](#), I saw an interesting opportunity to port the Google Gears [Database API](#) (note the Gear in the logo!)



If I could use the same API for both client and server side database access, then I can be enabled to do things like:

- Use one API, and have the system do a sync from local to remote databases
- If the user has JavaScript, use a local database, else do the work remotely
- Share higher level database libraries and ORMs such as [Gears DBLib](#) for use on server side data too

I quickly built a prototype to see if this would all work.

The [Jaxer shim of the Gears API was born](#), and to test it out I took the [database example from Gears itself](#) and made it work.

To do so, I only had to make a few changes:

Run code on the server

I changed the main library to run on the server via:

PLAIN TEXT

HTML:

```
1.
2. <script type="text/javascript" src="gears_init.js" runat=
3.
```

I wrapped database access in proxy objects, such as:

PLAIN TEXT

JAVASCRIPT:

```
1.
2. function addPhrase(phrase, currTime) {
3.   getDB().execute('insert into Demo values (?, ?)', [phr
4. }
5. addPhrase.proxy = true;
6.
```

This now allows me to run the addPhrase code from the browser, and it will be proxied up to the server to actually execute that INSERT statement.

This forced me to separate the server side code from the client side code, which is a better practice anyway, but it does make you think about what goes where. In a pure Gears solution I can put everything in one place since it all runs on the client.

Create the new gears_init.js

A new [gears_init.js](#) acts as the shim itself. Instead of doing the typical Gears logic, it implements the Gears database contract. This wasn't that tough, although there are differences between the Gears way, and the [Jaxer.DB](#) way. The main difference is to do with the ResultSet implementation, where Gears goes for a `rs.next()/rs.field(1)` type model versus the [Jaxer.DB](#) `rs.rows[x]` model.

I actually much prefer [Gears DBLib](#) as it hides all of that, and just gives the programmer what he wants... the rows to work on.

oncallback magic

In the current [Jaxer](#) beta, I ran into an issue where I wanted the Gears library to just "be there" for any proxy requests.

You have to think about the lifecycle of a [Jaxer](#) application, and the [documentation tells you what you need to know](#) to work around the issue.

In this case, I wrapped the code in:

PLAIN TEXT

JAVASCRIPT:

```
1.
2. function oncallback() {
3.   // create the wrapper here
4. }
5.
```

This is less than ideal, and Aptana is playing with nice scoping which would enable you to just say "hey, load this library once and keep it around for the lifetime of the server | application | session | page". That will be very nice indeed.

You can do a little bit of this by opening up your **jaxer_prefs** file and adding the resource for your file:

```
PLAIN TEXT

JAVASCRIPT:

1.
2. // This option sets up an html document that will be loaded
3. // everytime a callback is processed. This has to be a
4. // If not specified, an empty document will be loaded.
5. // pref("Jaxer.dev.LoadDocForCallback", "resource:///Firefox
6.
```

Future...

This is just the beginning. As I mentioned at the beginning, I am interested to see where you can take this to handle clients who do not support JavaScript, and also to deal with synchronization with minimal code (sync from local to remote with exactly the same SQL API).

Posted by [Dion Almaer](#) at 8:02 am

7 Comments

 3 rating from 21 votes

Saturday, February 2nd, 2008

Ajaxian Roundup for January, 2008: JavaScript Turtles and IE 8

Category: [Roundup](#) 

January has started the month out in some style. We are seeing a lot of news that shows the Web may actually be moving forward a little. One sign of that is the trickle of news that comes out of Redmond on IE 8. [IE8 Compatibility with X-UA-Compatible](#) sparked debate throughout the entire Web community, and when all is said and done, it shows you what happens when you do not have good early communication. Just work with us. Before you ship. Chat with Hixie and [get involved in HTML 5 now](#).

JavaScript continues to thrive and move throughout the stack. First we had the news of [Aptana Jaxer](#), which allows you to write your Ajax code and have it run on the server. This even means munging with the DOM and having it all work and spit out HTML. This isn't your old ASP JScript.

I also had the pleasure of chatting with Steve Yegge on [Rhino on Rails: JavaScript MVC on the server](#) where we discussed the implications of having a Rails port in JavaScript.

The browsers keep getting better, and we saw real support for [Cross-Site XMLHttpRequest in Firefox 3](#) and the like. I have been talking over [Google Gears future APIs](#) that may be in early stage work, or just ideas in my head. I think that we are getting the word out about Gears not being just about Offline, but a tool to upgrade the Web on the fly.

We are seeing more of the social networks getting into the mix. Facebook released [an Animation library](#) and then followed that up with a full [JavaScript Client Library](#).

And then Google just released the [Social Graph API](#).

A great month, and here is to the next one:

JavaScript

- [Aptana releases Jaxer, Ajax server built on Mozilla](#)
- [Rhino on Rails: JavaScript MVC on the server](#)
- [Secure String Interpolation in JavaScript](#)
- [DomAPI 4.5: New, improved, and more free](#)
- [Favicon access via JavaScript](#)
- [Graceful Degradation of Firebug Console Object](#)
- [Eloquent JavaScript](#)
- [Making Ajax Applications Scream on the Client](#)
- [Ajax.NET: Move to ASP.NET Ajax with PageMethods](#)
- [Dean Edwards IE7.js 2.0 Release](#)
- [Autotesting JavaScript in Rails](#)
- [Apache CouchDB: Congrats to IBM and Damian Katz](#)
- [JavaScript: It's Just Not Validation!](#)
- [window.onload: another solution to get it going](#)
- [EditArea: Rich Sourcecode Editor](#)
- [Survey on Ajax IDEs and development](#)
- [Beyond DOM](#)
- [Cross-Site XMLHttpRequest in Firefox 3](#)
- [XUL Templates](#)
- [Accessible Google Charts](#)
- [Hijoslide JS: JavaScript Thumbnail Viewer](#)
- [PBwiki JavaScript Testing](#)
- [Qooxdoo 0.7.3 Release](#)
- [Eclipse RAP Demonstration](#)
- [Debug apps with Drosera for Windows](#)
- [MooTools 1.2 Beta 2 Released, Adds New Element Storage](#)
- [The Art and Science of JavaScript Games](#)
- [Sexy JavaScript?](#)
- [Library Agnostic LightBox](#)
- [Facebook releases Animation library](#)
- [Facebook releases JavaScript Client Library](#)
- [Lily, JavaScript Visual Programming Tool](#)
- [Dreamweaver Users Rejoice! Support for JS Libs Now Available.](#)

Prototype

- [Prototype 1.6.0.2 security and performance improvements](#)
- [Prototype: new cheat sheet and in place editor](#)
- [Do you have a pretty date?](#)

Dojo

- [dojo.mojoe: parody of script.aculo.us homepage in Dojo](#)
- [Bunkai: Dojo Interactive Workbench](#)

Ext

- [Linsidadmin: Rails 2.0 Ext Admin](#)
- [Ext 2.0.1 Released, Community Projects Continue to Grow](#)

- [Ext Scaffold Generator Plugin for Rails](#)
- [Ext 2.0 and ColdFusion](#)
- [Simplicity: PHP Ajax Framework using Ext](#)
- [ExtTLD: Create Ext components with XML](#)

jQuery

- [jQuery Validation Plugin v1.2 Updated](#)
- [jQuery ScrollTo Plugin](#)

GWT

- [GWT Videos from GWT Conference Available](#)
- [Cool and useful GWT Solutions](#)

YUI

- [YUI Grid CSS and Grid Builder](#)
- [YUI ready to celebrate 2nd Birthday](#)

DWR

- [DWR/TIBCO GI Integration: qi.js](#)
- [Book: Practical DWR 2](#)

Gears

- [Google Gears Future APIs](#)
- [Buxfer: Another Personal Finance 2.0 Site](#)
- [XSTM: Shared Transacted Memory](#)

Flash / AIR

- [as3Query: jQuery support to ActionScript](#)
- [CommandProxy: Integrating AIR and .NET](#)
- [Adobe AIR is on Fire](#)

JSON

- [JSON 2.0: Libraries and browser support](#)
- [JsonSQL: JSON parser, SQL style](#)
- [JSONLib: JSON Extensions a la E4X](#)

Browsers / HTML Standards

- [IE8 Compatibility with X-UA-Compatible](#)
- [How IE Mangles the Design Of JavaScript Libraries](#)
- [The IE7 auto-rollback: fact and fiction](#)
- [John Lilly, CEO of Mozilla, Interviewed](#)
- [Mozilla hires Aza Raskin and other Humanized folk](#)
- [Full Page Zoom Is For Sissies](#)
- [Sub pixel fun with browsers](#)
- [HTML 5 Public Working Draft Released](#)
- [Getting HTML 5 styles in IE 7+](#)
- [Animated PNG in Firefox 3](#)
- [Microsoft JavaScript Memory Leak Detector](#)

CSS / Design

- [Javascript CSS Selector Engine Timeline](#)
- [Acid 3 and the future of memory leaks](#)
- [Effect.wobble and Effect.illuminate: iPhone effects in the browser](#)

Comet

- [Native Comet Support for Browsers](#)
- [20,000 Reasons that Comet Scales](#)

Security

- [Is using a JS packer a security threat?](#)
- [Book recommendation: Ajax Security by Hoffman and Sullivan](#)
- [XSS: Flash and Rails](#)
- [Dangers of Remote Scripting](#)

Editorial

- [Zed Shaw interview on Rails community, enterprise, Ajax, patents, and a whole lot more](#)
- [How widgets go viral](#)
- [A Study of Ajax Performance Issues](#)
- [You Used JavaScript to Write WHAT?](#)
- [Most Relevant Browsers of 2009](#)

Showcase

- [Triggitt: WYSIWYG Content Insertion Tool and Platform](#)
- [Drinking offline at the Happy Hour while being openly social](#)
- [Tastebook: Rails Ajax Cookbook App](#)
- [Domings: Changing the feedback model](#)
- [Protagonize: Choose your own Web 2.0 Adventure](#)
- [Mibbit: Ajax-based IRC Client](#)
- [iDesktop: YouTube Viewer](#)
- [Photophlow: Interactive Community Flickr](#)
- [CNN Politics Election Center](#)
- [AjaxSwing 2.0: AJAX front end for Swing applications](#)
- [AjaxRain.com Gets a Facelift](#)
- [Flemish Politics Election Center](#)
- [HTML Purifier 3.0](#)
- [Winter Holiday Christmas Lights](#)
- [Amaltas SVG Drawing Tool](#)
- [CuePrompter: Javascript Teleprompter](#)
- [Roxer: Simple web page creation and editing](#)
- [New Twist on Date Pickers](#)
- [Audience Measurement Demo](#)
- [O'Reilly Maker](#)
- [Less maintenance code tutorials with Ajax Code Display](#)

Posted by [Dion Almaer](#) at 12:58 am

[3 Comments](#)



4.7 rating from 9 votes

Wednesday, January 30th, 2008

Rhino on Rails: JavaScript MVC on the server

Category: [JavaScript](#) , [Java](#) , [Framework](#) , [Rails](#)

Cross posted from my personal blog

Last week we posted about [Jaxer](#) which offers an approach of [turtles all the way down](#) where JavaScript is used on the client and the server.

Then, I got to interview Steve Yegge. Last year, Steve posted about [Rhino on Rails](#), his port of Ruby on Rails to the JavaScript language on the [Rhino](#) runtime.

Ever since he [presented on the 'Google Rails Clone' at FooCamp](#) and he [posted about the internal Google Rhino on Rails project](#), people have been curious to learn more.

- What does it mean to port Rails to JavaScript?
- What can't you do since JavaScript doesn't have the same meta programming facilities?
- Rails = a group of "Active*", so did you re-implement everything?
- What do you gain out of having JavaScript all the way down?
- Does it actually make sense to have j.js? Server side JavaScript generating client side JavaScript? Argh!
- What is the state of Rhino?
- Will Rhino support JavaScript 2?

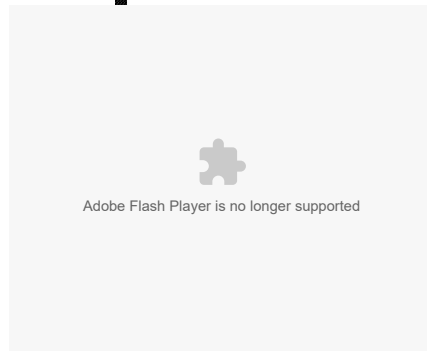
And of course, the big questions:

☞ When do I get to see it!

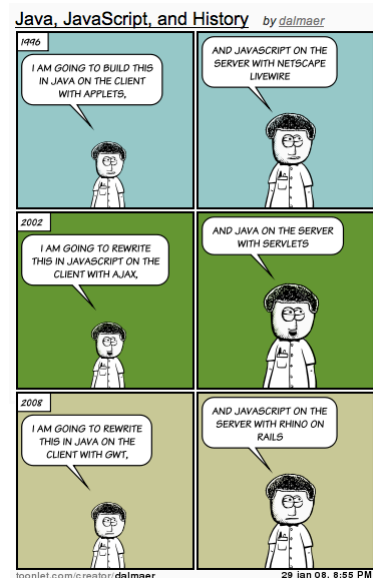
I happen to be in Seattle at the Google offices, so I was able to ask all of these questions and more. Steve was a fantastic host, and I really enjoyed chatting with him.

This is the kind of video I want to explore at Google. We have many great developers working on cool technology. I want to get them on camera, participating with the community when I can. Sometimes we can talk about products and APIs, but sometimes we will talk about fun ideas and projects that we are working on such as Rhino on Rails.

Anyway, give it a watch and let me know what you think:



This also lead me to [the fun idea of Java and JavaScript flip-flopping](#):



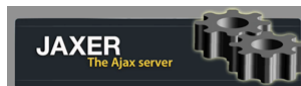
Posted by [Dion Almaer](#) at 8:20 am
[12 Comments](#)
4 rating from 29 votes

Tuesday, January 22nd, 2008

Aptana releases Jaxer, Ajax server built on Mozilla

Category: [Ajax](#), [JavaScript](#), [Screencast](#), [Library](#), [Server](#), [Aptana](#)

Aptana has been known for its Eclipse based Ajax IDE [Aptana Studio](#). Paul Colton, CEO, has impressed us at The Ajax Experience as he has shown of Studio, and how Aptana is fast to adapt and come out with support for iPhone development, Adobe AIR, and more.



But today Aptana is breaking out of the IDE world, and joining the server market with the release of [Jaxer](#), which they are calling an "Ajax Server". **Jaxer** brings JavaScript, the DOM, HTML, CSS, to the server as it tries to unify the development model for developers.

You can think of the server as a Mozilla version of Tomcat. It plugs nicely into Apache (and more in the future), and handles the web application requests for

you.

An example: validation

One solid way to understand **Jaxer** is to look at an example such as [validation \(screencast\)](#). Wouldn't it be nice to be able to reuse your validation logic on the client and the server? This example shows you how you do just that:

Part 1: Server Proxy

This code shows you how to use `runat="server-proxy"` to wrap that code in a way that the client can call it, but it gets seamlessly ran on the server:

```
PLAIN TEXT
HTML:
1.
2. <script runat="server-proxy">
3.     // Set the value of the input field 'name'
4.     // to the initial value
5.     document.getElementById("name").value =
6.         Jaxer.File.read("name.txt") || "A Long E
7.
8.     // Function to save the value to a file
9.     // once validated
10.    function saveToFile(value) {
11.        validate(value);
12.        Jaxer.File.write("name.txt", value);
13.        return new Date();
14.    }
15. </script>
16.
```

Part 2: Share code with 'both'

Then you write a simple validator that can be run on both client and server:

```
PLAIN TEXT
HTML:
1.
2. <script runat="both">
3.     // Use the same function to validate on the brow
4.     // for user feedback and on the server for secur
5.     function validate(name) {
6.         if (name.length> 5)
7.             throw new Error("'" + name +
8.                 "' exceeds 5 characters!");
9.     }
10. </script>
11.
```

Part 3: Client side script

Finally you have the client side code that calls in:

```
PLAIN TEXT
HTML:
1.
2. <script>
3.     // Browser-side script
4.     function save(){
5.         // Runs when user clicks save
6.         var name = document.getElementById("name
7.
8.         try {
9.             // Validate in browser, if succee
10.            // save on server
11.            validate(name);
12.
13.            // Call server-side function 'sa
14.            // and get return value
15.            var savedDate = saveToFile(name)
16.
17.            // Update the confirm DIV browse
18.            document.getElementById("confirm
19.                "Saved on " + savedDate;
20.        }
21.        catch (e) { alert(e.message); } // Show
22.    }
23. </script>
24.
```

This is the tip of the iceberg. There are other examples available as [part of the download](#), and as [screencasts](#).

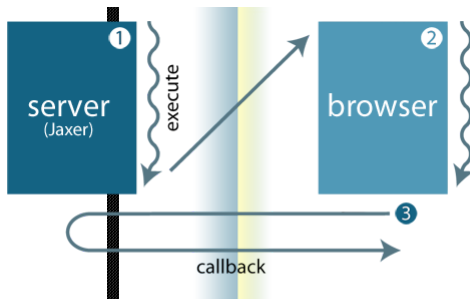
What does this mean?

If you don't like context switching between the Web world of HTML/CSS/JS/DOM (Ajax) and the server side language of your choice, then this could be the programming model for you.

Unobtrusive Ajax

There are some interesting benefits, such as the ability to do a good job with certain accessibility without much work. If the browser doesn't have JavaScript turned on, the server can run the code and produce the HTML for you. Obviously, the HTML interface that is returned would be less dynamic.

The execution flow looks like this:



- 1 Page executes on server and resulting page is sent to browser
- 2 Browser executes resulting page
- 3 Browser calls server asynchronously for new information

Prototype/jQuery/Dojo/... on the server

You can use your client-side library of choice on the server (e.g. Prototype, Dojo, jQuery, etc). My screencast below in fact does just that. The aim isn't to create a totally new programming model, but to rather be part of the Web. That being said, they had to add a set of libraries to allow you to do things on the server that you can't on the client. For example, the file reading in the example above:

```
Jaxer.File.read("name.txt").
```

Databases

To build server side Web applications you normally need to persist data. Database access is provided by `Jaxer.DB.*`, and SQLite is built in. As soon as I saw this, I pictured a wrapper for **Jaxer** that would bridge the Google Gears database API. There is the ability to build some simple syncing, and have the framework do the right thing depending on if you are offline or online, and shifting data between the local and remote SQLite database. Very promising.

Cross domain XMLHttpRequests

Since you can have XMLHttpRequests which are really coming from the server, you can talk to any domain. Again, in my example, I proxy through to Twitter, which allows me to do a Greasemonkey type solution that works in all browsers.

JavaScript 1.8

Since the latest Spidermonkey is available on the server, you can write [JavaScript 1.8](#) using new features such as *expression closures*, generators, and more. You have to be careful here and remember that this will only work for code ONLY running on the server. For all other code, you have to do the usual thing and test it with the various browsers that matter to you.

Rails for JavaScript?

I hope that we do not see a lot of huge pages with `runat="server"` all over the shop. We did that in the ASP/PHP/JSP of old. I am sure that frameworks will pop up to do MVC nicely with **Jaxer**. One option would be having **Jaxer** work nicely with projects such as Trimpath Junction and the project that Steve Yegge often talks about.... which is Rails-like but for JavaScript (using Rhino). Integration will also be an issue for a certain class of applications. I can imagine there being a particularly strong connection for something like Java.

IDE independence

Although Aptana Studio has top notch support for **Jaxer** (it better!), the product does not depend on any IDE and you are free to use it with anything. To show this, I use the stand alone **Jaxer** server and write my code using vi.

In fact, let's check out the video that builds a Twitter proxy in a few lines of code, showing how the beast works.

(I recommend watching the video [at Vimeo](#) to make a larger window to see the type).

Posted by [Dion Almaer](#) at 3:00 pm

[24 Comments](#)

★★★★☆
4 rating from 54 votes

[← Next Page →](#)

Powered by [WordPress](#)

[Entries feed](#). Valid [XHTML](#) and [CSS](#)

All rights reserved, Copyright 2008. [About Us](#) [Privacy Policy](#)